

5.2 Pitch Deck

10 slide · 5 phút · Đội **CLGT-EAM** — Nguyễn Đình Chương · Đào Văn Linh · Đồng Huỳnh Giao
Mỗi slide: **tiêu đề** + nội dung trên slide + lời thoại. Giữ chữ trên slide ở mức tối thiểu; phần giải thích nằm ở lời thoại.

Slide 1 — The Problem

Tiêu đề: Một ticket, ba lần gõ lại

Trên slide

Requirement —gõ tay→ Test case —gõ tay→ Azure DevOps —gõ tay→ Playwright spec

|

→ FAIL ở dòng đầu "Nhờ AI viết thẳng" → code trông hợp lý

- ~4.5 giờ / ticket · phần cần chuyên môn chỉ chiếm ~25%
- Bảo trì automation tốn hơn viết mới
- AI đoán selector → output phải viết lại

Lời thoại: "QC/QA gõ lại cùng một nội dung ba lần cho mỗi ticket. Nhờ AI viết thẳng thì không dùng được — model đoán selector từ ngữ cảnh nó nhìn thấy, code trông hợp lý và fail ở dòng đầu tiên. Và ở lớp dưới còn một vấn đề thứ hai: mỗi công cụ AI nội bộ tự giữ tài khoản, PAT và credential riêng."

Slide 2 — Target User

Tiêu đề: Hai người dùng, hai bài toán khác nhau

Trên slide

QC/QA Engineer	Tech Lead / Admin
8–15 ticket / sprint	Quản lý quyền và chi phí
Biết đọc code, viết Playwright cơ bản	PAT nằm trong .env từng máy
Cần: coverage đủ, spec không đở vật	Cần: một chỗ cấp và thu hồi, nhìn được chi phí

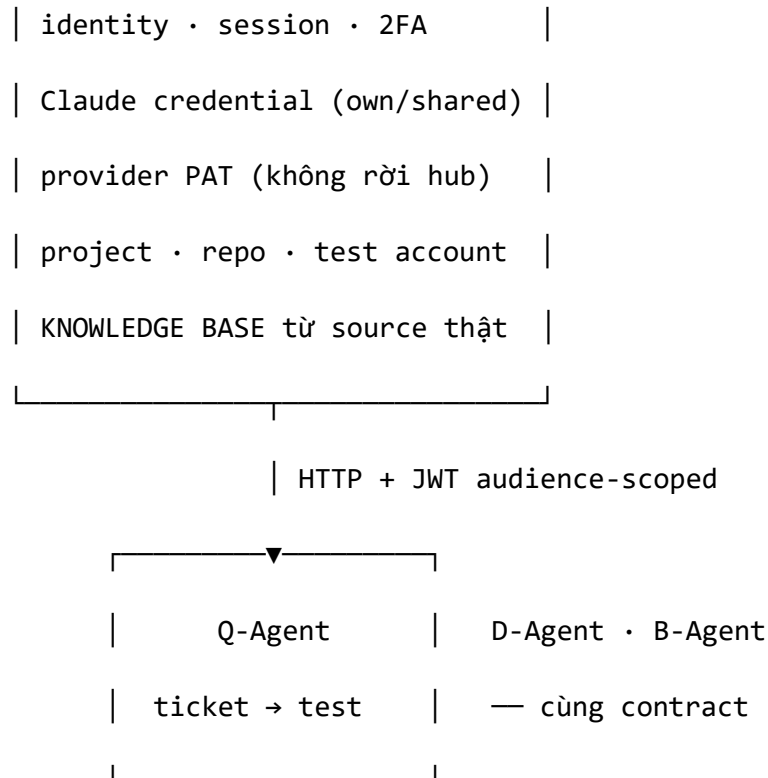
Lời thoại: "Người dùng chính là QC/QA. Người dùng thứ hai là tech lead — người chịu trách nhiệm khi có người vào hoặc rời team, và hiện đang phải đi thu hồi quyền ở nhiều nơi."

Slide 3 — Our Solution

Tiêu đề: Một hub, nhiều agent, một contract

Trên slide

┌────────── EmeHub ─────────┐



Lời thoại: "EmeHub giữ mọi thứ dùng chung và **không làm việc chuyên môn**. Q-Agent làm việc của QC/QA. Agent thêm vào sau gắn qua cùng một contract, chiếm một path segment mới trên cùng địa chỉ — không phải dựng lại nền tảng."

Slide 4 — How It Works

Tiêu đề: Pipeline 7 bước, 3 gate do người giữ

Trên slide

[1] Analyze → [2] REVIEW ★ → [3] Link ★ → [4] Automation

[7] Publish ★ ← [6] Evidence ← [5] Execution ←

★ = con người ký duyệt, chặn thật

- Case sinh ra ở pending; chỉ case approved đi tiếp
- Link có local mode (dry-run)
- Comment preview trước khi gửi

Lời thoại: "Bảy bước. Ba trong số đó con người phải ký duyệt, và ba gate này chặn thật chứ không phải cảnh báo. Chúng nằm đúng ở ba chỗ có hậu quả ra ngoài hệ thống."

Slide 5 — Why AI

Tiêu đề: AI ở đúng ba chỗ, và không ở chỗ nào khác

Trên slide

Dùng AI	Vì sao không deterministic được
Đọc requirement → test case truy vết tới AC	Requirement không có schema
Đọc codebase → route, selector, page object	AST cho biết có <input>, không cho biết đó là ô mật khẩu của luồng đăng nhập
Đọc log đồ → test sai hay product sai	Bài toán suy luận, và là chỗ QA tốn thời gian nhất

KHÔNG dùng AI: identity · encryption · JWT · phân quyền · gọi provider API

Lời thoại: "Ba việc này không có lời giải deterministic. Phần còn lại của hệ thống chúng tôi cố tình **không** dùng AI — nó là code thường, có test, và phải deterministic."

Slide 6 — Product / Demo

Tiêu đề: Magic moment — chạy thật trước, sinh spec sau

Trên slide

Cách thông thường: đọc source → ĐOÁN selector → sinh spec
→ FAIL → heal

live-harness: mở Chrome đã đăng nhập

 → thực thi TỪNG STEP trên app thật

 → resolve element qua accessibility tree

 data-testid → ARIA role+name → label →

CSS

 → dispatch trên CHÍNH selector sắp emit,

verify URL đổi

 → tạo test data nếu thiếu

 → CHẠY HẾT VÀ PASS RỒI MỚI EMIT SPEC

*Placeholder gate: selector bja · TODO · URL placeholder → bLocked **trước** khi chạy*

Lời thoại: "Đây là chỗ khác biệt lớn nhất. Q-Agent không sinh script rồi mới chạy — nó chạy thật trước, rồi mới sinh script từ đúng những gì đã chạy được. Selector trong spec không phải suy đoán; nó vừa được bấm xong."

(Chuyển sang demo trực tiếp hoặc video ở đây.)

Slide 7 — Impact

Tiêu đề: Hiệu quả không đến từ "AI gõ nhanh"

Trên slide

Nguồn hiệu quả	Cơ chế
Xoá bước gõ lại	Dữ liệu đi thẳng requirement → case → ADO → spec → comment
Đầu vào tốt hơn	AI làm việc trên knowledge base của repo thật
Bảo trì chuyển sang máy	Page object dùng lại + heal có grounding DOM
Khép vòng về ticket	Evidence và comment tự về work item

Đo được ngay trong sản phẩm: **cost / run · token / user · failure class · AC coverage · first-run green rate**

Lời thoại: "Hiệu quả không nằm ở chỗ AI viết nhanh hơn người. Nó nằm ở chỗ output không phải làm lại. Và mọi con số này đo được ngay trong sản phẩm — chi phí hiển thị theo từng run."

Slide 8 — Differentiation

Tiêu đề: Bốn thứ mà một "AI test generator" thường không có

Trên slide

	Cách thường thấy	Ở đây
Grounding	Prompt mô tả chung chung	project-bootstrap đọc toàn bộ source thật; KB giàu lên bằng selector verified-at-runtime
Thứ tự	Sinh spec → chạy → heal	Chạy → verify → mới sinh spec
Guardrail	Không có	Placeholder gate · failure class 5 loại · bounded run
Nền tảng	Mỗi tool một silo	Một identity, một credential store, một KB cho mọi agent

Bảo mật cưỡng chế bằng kiến trúc: PAT không rời hub · Claude credential ra qua **một** hàm · run-scoped grant chỉ reach **3 route**, giới hạn bằng wiring chứ không bằng điều kiện trong code · **không fail-open**

Lời thoại: "Điểm cuối đáng nói riêng: giới hạn của credential grant không được cưỡng chế bằng một câu lệnh điều kiện, mà bằng chính cách wiring — không route nào khác trong hub phụ thuộc vào nó. Thêm một route mới không vô tình mở rộng phạm vi."

Slide 9 — Roadmap

Tiêu đề: Đang ở đâu, đi tiếp thế nào

Trên slide

Đã CHẠY EmeHub đầy đủ (788 test) · Q-Agent 7 bước · SSO hub → Q-Agent

Knowledge base build trên hub · một origin cho cả suite

CHƯA XONG Q-Agent mới dùng identity từ hub.

Credential và project vẫn còn bản của agent.

TIẾP THEO P1 cutover identity → P2 cutover credential → P3 cutover project + KB

P4 eval dataset đóng gói → P5 RS256 + JWKS

SAU ĐÓ D-Agent (DEV) · B-Agent (BA) – hub side đã sẵn sàng, việc còn lại ở phía agent

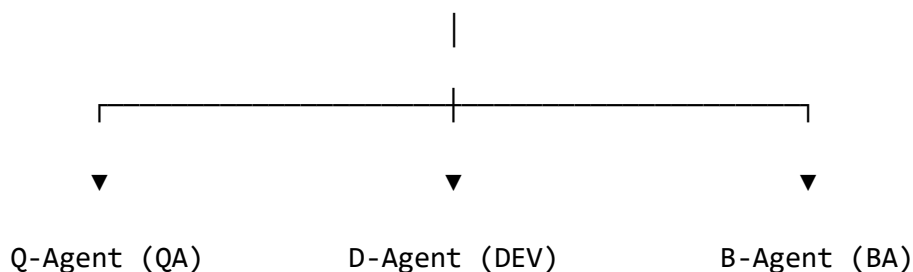
Lời thoại: "Nói thẳng để không bị hiểu nhầm: hai agent chưa chuyển hẳn sang dùng hub. Hôm nay Q-Agent mới dùng identity. Đó là việc tiếp theo, và cũng là lý do phần nền tảng được xây trước."

Slide 10 — Vision / Closing

Tiêu đề: Agent thứ ba không phải dừng lại gì cả

Trên slide

Một login · một chỗ khai credential · một knowledge base



Ba điều kiện để gọi là production

1. Không agent nào còn authentication riêng
2. Không agent nào còn bản sao credential
3. Agent thứ ba onboard **chỉ bằng cách đọc contract** — không PR nào vào hub

Lời thoại: "Thước đo thành công của hub không phải là màn hình đẹp. Nó là: khi agent thứ ba xuất hiện, có PR nào phải vào hub không. Nếu không có, thì hub đã làm đúng việc của nó."

Phụ lục — câu hỏi hay gặp

Câu hỏi	Trả lời ngắn
---------	--------------

Dữ liệu có ra ngoài không?	Toàn bộ self-hosted. Ra ngoài duy nhất là lời gọi tới Anthropic API.
Nếu AI sinh test case sai?	Review Center là gate bắt buộc; mapping AC → case cho thấy chỗ hở.
Nếu heal làm spec "xanh" giả?	Heal bounded 3 lần, chỉ chạy khi có grounding DOM; failure class hiển thị để người phản bác. product_defect không được heal.
Chi phí LLM kiểm soát thế nào?	Cost ceiling + turn cap + timeout mỗi run; token và cost ghi theo từng người.
App có SSO/MFA thì sao?	Local Agent chạy trên máy tester; cookie và storageState không rời device.
Số liệu hiệu quả đã đo chưa?	Cơ chế đo đã có sẵn trong sản phẩm; cỡ mẫu ticket còn nhỏ nên đội không quy thành phần trăm. Chi tiết ở tài liệu AI Evaluation.
Vì sao không dùng RAG / vector store?	Câu hỏi ở đây không phải "đoạn code nào giống câu hỏi này" mà "route và selector của project này là gì" — một fact table có cấu trúc trả lời tốt hơn top-k chunk.